System Programming Report

Java Chat System

Toby Aldred

N1379395

1.0 Design and Architecture

1.1 The Problem:

The aim of this project is to develop a java-based network chat system capable of supporting communication between multiple clients simultaneously, the system will use socket programming to establish and maintain connections, demonstrating how distributed applications exchange data in a live practical environment, to achieve this the server must be able to:

- Accept incoming socket connections from multiple users at the same time.
- Maintain active, persistent communication between each clients connected channels.
- Relay messages efficiently and reliably across the server client network.

The core problem being addressed is how to design a system that can manage concurrent connections without experiencing performance issues, message loss, or blocking behaviour, while ensuring that the communication between client – server remains secure and controlled using socket programming.

1.2 The Security Aspect:

Because the system requires for users to transmit messages across a network, security becomes a crucial requirement. The design prioritises:

- Confidentiality – ensuring that messages cannot be intercepted, viewed, or accessed by unauthorised users.
- Integrity – ensuring that messages are not altered, corrupted, intercepted, or manipulated by malicious actors during data transmission.

To meet these goals the system must incorporate secure communication practices such as encryption, controlled access, and input validation techniques. These practices help protect both the users and the data being exchanged, ensuring that the chat remains trustworthy, even when multiple clients are connected simultaneously.

1.3 Key Features / Requirements of the System:

The system has been designed to provide a secure, dependable, and scalable connection between multiple clients. To achieve this, the implementation incorporates a combination of core networking features, security mechanisms, and advanced programming techniques. In a combination these features demonstrates both functionality and a better understanding of system level programming in java. The table below highlights the key features of the system. Containing additional enhancements that will be added to the system.

Category	Feature	Technical Implementation	Design Choice UX and Complexity
Networking	Multi-Client architecture	Java Executor Service (Thread Pool)	Improves resource management by capping active threads, preventing server crashes during high traffic. Recycles active threads rather than creating infinite new ones
Networking	TCP Socket Control	ServerSocket and Socket classes.	Chosen for connection-oriented reliability to ensure zero packet loss during chat transmission.
Networking	Parallel Messaging	Synchronized routing for Private/Group chats.	Ensures parallel conversations remain isolated, preventing data leakage between users.
Networking	Active User Management	Concurrent HashMap for thread-safe tracking.	Allows lookups for routing messages while ensuring thread safety in a concurrent environment.
Networking	Active Users	Iterate over ConcurrentHashMap	Allows users to see who is online at given time. Safely retrieves live Threadpool mapping without locking the system and presents user with who is online.
Security	Secure Authentication	SHA-256 Hashing with Salt.	Mitigates data breach risks; salt ensures identical passwords have unique hashes, stopping rainbow table attacks.
Security	Account Lockout	Defensive counter logic for failed attempts.	Specifically addresses the requirement to mitigate potential security threats like brute-force attacks
Security	AES Encryption	Symmetrical encryption of message payloads.	Ensures Confidentiality; even if packets are intercepted, the content remains unreadable without the key.
Security	Admin Kick	Target User and force disconnect from socket	Moderation to keep chat safe, regular users do not have ability to. Cross thread communication from admin to server to kill terminal and disconnect of target user
Monitoring	Chat History Logs	Persistent .txt file I/O streams.	Provides a searchable record for user reference and system troubleshooting.
Monitoring	User activity Logs	Timestamped connection/disconnection tracking.	Essential for accountability and monitoring the live practical environment" of the server

Monitoring	Security Event Logs	Dedicated logging for failed logins/lockouts.	Supports security auditing and helps identify malicious activities within the network.
Monitoring	Admin log retrieval	LogManager.getChat Logs	Gives admins /moderators historical context when logging in. loads the csv into memory and securely transmitted to admin via AES over socket.

1.3.1 System Architecture Justification

The systems architecture has been chosen in this format as it isolates the systems responsibilities, into a modular, independent class structure which is able to support scalability and maintenance. This ensures the following:

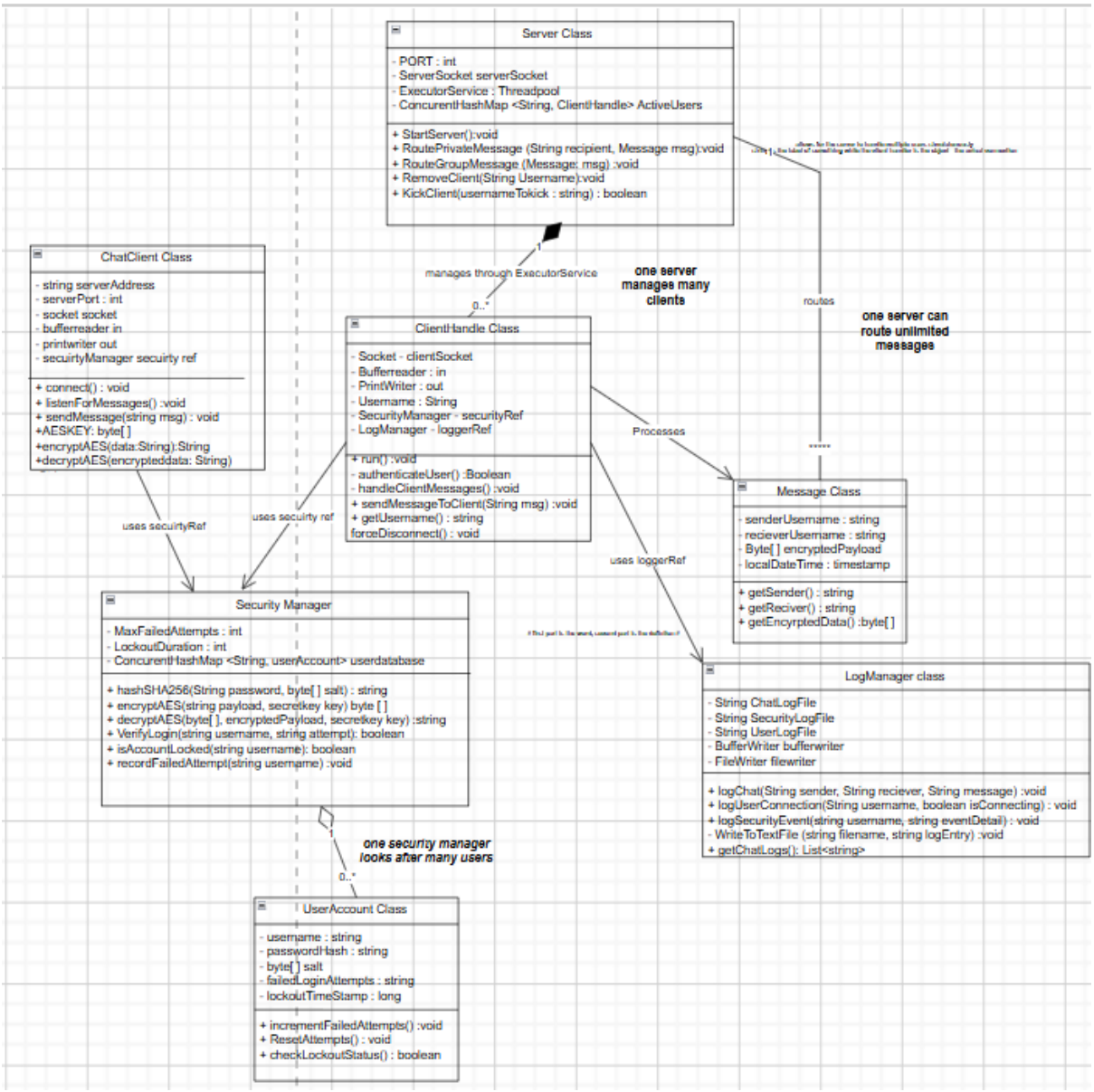

Figure 1: Features the Class diagram.

Modular Security & Accountability: The Security Manager class encapsulates all sensitive logic—including SHA-256 password hashing and AES encryption. This ensures that security protocols are managed consistently without bloating the main server. Similarly, the Log Manager centralises activity recording, which is essential for the security auditing and troubleshooting aspects.

Concurrent Resource Management: To manage multiple clients simultaneously, the system uses the Java ExecutorService (Thread Pool) to limit active threads and prevent exhaustion of the systems resources. This is supported by a HashMap for active user tracking, ensuring data safety and routing performance.

1.3.2 Algorithm Flowchart and Logic Justification

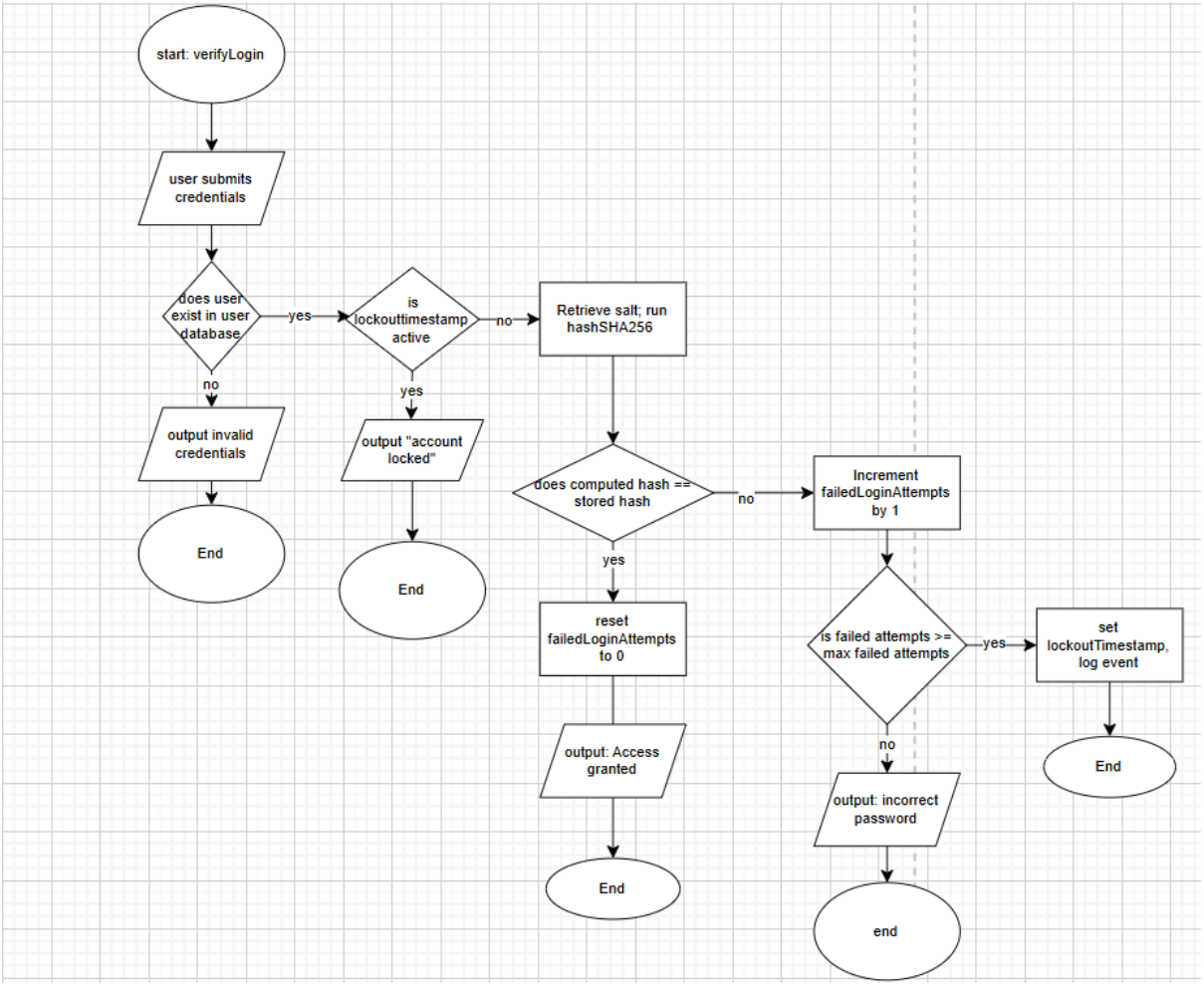

Figure 2: Features the Algorithm flowchart.

The algorithm flowchart (Figure 2) details the operational logic for the verify login method within the security class. Security being a crucial requirement for this system, the design prioritises confidentiality and integrity by implementing a robust, multistage authentication gateway.

Key features:

- **Defensive Authentication**: The system specifically addresses malicious activity, such as brute force attacks. This is done by using defensive programming and counter logic.
- **State based security:** The systems algorithm uses a failed attempts counter and a lockout time variable to manage user access.
- **Cryptography integrity:** to protect user data, the process incorporates SHA-256 Hashing with salt. This design and the use of cryptography ensures that identical passwords produce unique hashes, which mitigate the rainbow table attacks.
- **Operational efficiency:** By checking the lockoutTimestamp before performing cryptographic operations, the system minimises resource exhaustion during unauthorised access attempts, which ensures a dependable and scalable connection for the users.

2.0 Implementation

2.1 Concurrent Networking and Resource management

The server uses the Java ExecutorService to manage a fixed Threadpool, which is essential for a multi-client system.

```
public class Server {
    // pool of ten threads at a time
    private static final int MaxClients = 10;
    private static ExecutorService pool = Executors.newFixedThreadPool(MaxClients);

    //storing username and thread handling specific user
    public static ConcurrentHashMap<String, ClientHandle> activeUsers = new ConcurrentHashMap<>()
    Run|Debug
    public static void main(String[] args) {

        try (ServerSocket serverSocket = new ServerSocket(port: 6666)){ // declaring in the try b
            System.out.println(x: "Server started. Waiting for a client...");

            // server loops continiously to accept new connections
            while (true){
                Socket clientSocket = serverSocket.accept();
                System.out.println(x: "A new client Connected!");

                ClientHandle clientThread = new ClientHandle(clientSocket);
                pool.execute(clientThread);
            }
```

Figure 1: Multiclient Architecture

This solves the problem of system resource exhaustion and improve processing capability in multithreaded environments (Yinbo) by using a fixed thread pool the server caps the number of active threads, which prevents a malicious actor or a spike of traffic from crashing the system.

Meanwhile The use of the concurrentHashMap for active user tracking builds thread safety, unlike the standard HashMap it allows for multiple threads to perform operations simultaneously without causing memory corruption, crashes or breaking the entire system which is crucial for parallel messaging which the system is dependent on.

2.2 Cryptographic security for data integrity

The system prioritises confidentiality and data integrity by separating authentication hashing and message encryption.

2.2.1 SHA-256 Hashing with salt

```
// Generates a random cryptographic salt
public static byte[] generateSalt() {
    SecureRandom random = new SecureRandom();
    byte[] salt = new byte[16];
    random.nextBytes(salt);
    return salt;
}

// Hashes the password with the salt using SHA-256
public static String hashSHA256(String password, byte[] salt) {
    try {
        MessageDigest md = MessageDigest.getInstance(algorithm: "SHA-256");
        md.update(salt);
        byte[] hashedBytes = md.digest(password.getBytes());
        return Base64.getEncoder().encodeToString(hashedBytes);
    } catch (Exception e) {
        throw new RuntimeException(message: "Hashing error", e);
    }
}
```

Figure 2: Cryptographic Hashing with salt

The storing of passwords as plain text is a critical security liability. The system uses SHA-256 Algorithm to strengthen the security of the system. The system initially generates a unique 16-byte random salt for every user that signs in stored within a file. This ensures that identical passwords result in completely different hashes in the stored file. Salting each password with a unique random value before SHA-256 hashing is known to render rainbow table attacks ineffective and significantly improve password security within a system (Patra, R., and Patra).

2.2.2 AES Encryption

```
// secret key for AES-128 encryption
private static final byte[] AESKEY = "CourseworkKey123".getBytes();
public static String encryptAES(String data){
    try{
        SecretKeySpec secretKey = new SecretKeySpec(AESKEY, algorithm: "AES");
        Cipher cipher = Cipher.getInstance(transformation: "AES");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);
        byte[] encrypted = cipher.doFinal(data.getBytes());
        return Base64.getEncoder().encodeToString(encrypted);
    }catch (Exception e){
        System.out.print("Encyrption Error" + e.getMessage());
        return null;
    }
}
```

Figure 3: AES Encryption & Cipher – Encrypt Content

User confidentiality is maintained by encrypting the message payload before it is transmitted to the receiver over the TCP socket. Using AES-128 the encryption ensures that even if network packets are intercepted the interceptor only sees incomprehensible data because it lacks the key, demonstrating confidentiality against interception (Karacali, Donum et.al) which only the sender and the intended receiver have. This implementation transforms standard text into Base64 encoded ciphertext which ensures that the data is transmitted safely without being corrupted. Additionally, AES is considered one of the most efficient and resistant to brute-force attacks. (Abbas, Joshua et.al).

2.3 Defensive Programming

```
String attemptHash = hashSHA256(passwordAttempt, user.getSalt());

if (attemptHash.equals(user.getPasswordHash())) {
    user.resetAttempts();
    return "SUCCESS";
} else {
    user.incrementFailedAttempts();
    if (user.getFailedLoginAttempts() >= MAX_FAILED_ATTEMPTS) {
        user.setLockoutTimestamp(System.currentTimeMillis() + LOCKOUT_DURATION);
        return "ACCOUNT_LOCKED";
    }
    return "INVALID_PASSWORD";
```

Figure 4: Defensive Programming – Lockout Mechanism

The system contains an authentication gateway for the login system, within this containing a lockout mechanism to mitigate brute force attacks.

Furthermore, the system demonstrates defensive programming my managing the security state of each user account being newly created or stored accounts in Users.csv. By comparing the current system time in milliseconds against a stored lockoutTimestamp the system can dynamically grant or deny access. This approach is efficient as it checks the account status before performing any computationally expensive cryptographic hashing, minimising resource exhaustion during unauthorised access attempts without blocking legitimate users (Olakanmi, Odeyemi).

2.4 Monitoring and Accountability

Each event that happens within the system is recorded in a persistent CSV log file through LogManager. Providing an audit trail is essential evidence for detecting malicious activity, troubleshooting, and supporting security and forensic audits. (Ali, Ahmed et.al)

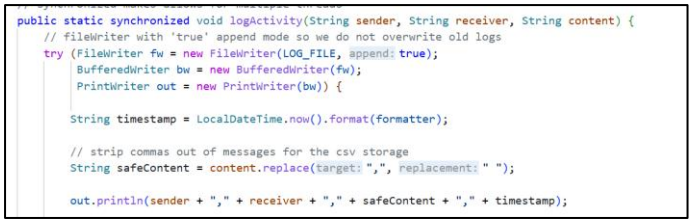

Figure 5: Log writing via CSV

The implementation of the synchronised keyword is essential in a multi thread environment. Because multiple ClientHandle threads may attempt to record messages or security events at the same time, there is a risk of a race condition which means data could be corrupted or unaccounted for. The synchronised modifier ensures that only one thread can access the writer at a time to mitigate this risk.

Furthermore, the code implements sanitisation of code, this means that all commas accidentally inputted are replaced with spaces, which ensures that the CSV remains consistent and readable. Cleaning user input is critical to prevent malformed or malicious data and to maintain data integrity (Asabashvili)

2.5 Administrative Controls

The ClientHandle class manages the lifecycle of a connection for each client connected. This also includes the execution of admin commands and closure of resources.

Figure 6: Admin Control + Force Quitting

The Figure 6 demonstrates identity-based control and a case-insensitive check to verify if the current username matches the "admin" profile in the users.csv before allowing access to the kick client method, which prevents unauthorised users from disrupting the network, any user that attempts to use the admin command is presented with a message saying they do not have permissions and no action is taken upon that.

2.6 Resource Cleanup

```
//clean up resources - force closes everything - stops everything from crashing
try {
    if (in != null) in.close();
    if (out != null) out.close();
    if (clientSocket != null) clientSocket.close();
}catch (IOException e) {
    e.printStackTrace(); // tells you where the problem is @w3schools
}
```

Figure 7: Resource cleanup

The resource cleanup in the finally block in Figure 7 guarantees that the socket and the I/O streams are closed even if there is an unexpected exception that occurs that force closes the chat session, this is crucial for the ExecutorService Threadpool. It ensures that the thread is successfully returned to the pool and the user is removed from the concurrentHashMap freeing up the thread in the Threadpool for a new client to join.

2.7 Parallel and Private Messaging

```
if (decryptedMessage.trim().startsWith(prefix: "/msg ")) {
    String[] parts = decryptedMessage.trim().split(regex: "\\s+", limit: 3);

    if (parts.length == 3) {
        String targetUser = parts[1];
        String privateMessage = parts[2];

        LogManager.logActivity(username, targetUser, privateMessage);

        String formattedPrivate = "(Private) " + username + " whispers: " + privateMessage;
        boolean success = Server.routePrivateMessage(SecurityManager.encryptAES(formattedPrivate), targetUser);

        if (!success) {
            out.println(SecurityManager.encryptAES("Server: User '" + targetUser + "' is not online."));
        } else {
            out.println(SecurityManager.encryptAES("You whispered to " + targetUser + ": " + privateMessage));
        }
    } else {
        out.println(SecurityManager.encryptAES(data: "Server: Invalid command format. Use: /msg username message"));
    }
    continue;
```

Figure 8: Private/ Parallel messaging Encryption/ Decryption

The main feature of this system was the group messaging, however as it developed the design and the implementation of private messaging via /msg, a hard coded command that allows for clients to directly message anyone on the server in the format of /msg username content.

To achieve this string manipulation was needed, the expression trim.split(\\s+) split the message into exactly three parts: /msg username content which ensured that the system could manage various amounts of white spaces through input validation techniques (W3schools 2026). Ensuring that private conversations remain encrypted and isolated demonstrating the systems scalability. However, clients' whispers are passed to the Log Manager which is essential for identifying malicious activities within a network.

2.8 Route targeting

Figure 9: Routing to target user.

The delivery of the private message from client to client is managed by a method located in Server.java. by using the get() operation and using the target username the server identifies the specific ClientHandle being used for the client (the dedicated thread) and pushes the encrypted message to their output stream. Before the message is routed it is passed through the SecurityManager.encryptAES (W3schools 2026) method which ensures confidentiality is maintained during transmission.

3.0 Implementation Challenges

3.1 Challenge 1: Data Corruption in CSV Logs

Storing user messages in the csv meant that if a user has typed a comma in their chat message it would break the csv file structure and corrupt the log data continuously and shift all the data into the wrong column of the csv and create one large mess.

I solved this issue by implementing input sanitisation (Happycoding.io) where " , " was replaced by " "this was handled by the LogManager. Additionally similar logic was applied for when clients may input a " , " in their username, doing this early on ensures that long term data integrity is maintained.

3.2 Challenge 2: Corrupting User database with raw bytes

During the setup of the password security aspect of the system, there was a massive issue when file saving, the system originally only "generated 6 bytes" of random cryptographic salt – however this issue was quickly identified when the system would not function. Then after solving this the system generated 16 bytes of random cryptographic salt to protect user passwords. But because it was made of raw memory bytes , when trying to write it to the User.csv the file just corrupted the text and made the entire file unreadable. My initial thought to solve this was to save it as a regular string but then that avoided the whole point of the hashing process.

I ended up solving this by researching new methods in how to do this online, when I discovered "Base64" encoding. Before saving to the file my code now translates the raw bytes of data into a safe and text format using Base64.getEncoder().encodeToString(salt). When the server goes to load the file now it just decodes It back into bytes. This process took a lot of trial and error and a whole of researching techniques and trying to apply it however gave me more insight about how memory and file I/O coordinate together, this was probably one of the hardest aspects of the system to create.

3.3 Challenge 3: Client freezing

In a regular program using java calling a scanner to read the keyboard input completely pauses the program until the user presses enter (blocking behaviour) because of this a user would not see new incoming messages from the server if they were in the process of typing something themselves. To avoid this, I had to implement multithreading in Client1.java. To achieve this, I created a background runnable listener that always listens, which constantly uses a BufferedReader and decrypt incoming messages from the socket. Allowing for the main thread to be completely free to handle the scanner keyboard input- which allows for the chat to run in real time.

4.0 Testing and Evaluation

To ensure the validity of the tests shown in the test table below, all tests were conducted in the same controlled and local environment. To do this Server.java was hosted on a localised machine, using the address 127.0.0.1 on port 6666. To simulate the environment and test correctly. Up to 5 separate clients were executed simultaneously demonstrating the ExecutorService concurrency, this was accomplished by using multiple terminal windows- further allowing for a real time networking simulation and monitoring of thread allocation, parallel messaging, and messaging routing without interference.

4.1 Test data and results

Variable/ Data item	**Value**	**Expected Result**	**Actual Result**	**Action Taken**
Failed Login Counter	3 (Boundary)	System sets lockoutTimestamp	“ACCOUNT LOCKED” message returned access denied	n/a
Chat Message Input	“Hello, how are you” (invalid data)	Message broadcasts normally but is saved safely to the CSV log without breaking columns.	Traceback error/CSV Corruption: The comma broke the CSV delimiter, shifting data in ChatLogs.csv.	**Redesigned** LogManager code to use content.replace(",", " ") function
Username Input	“Test,234" (Illegal)	System refuses connection to protect database integrity.	"Invalid username. Commas not allowed." Connection dropped	n/a
Username Input	"User1" (Valid)	System accepts the username and proceeds to password check.	username accepted	n/a
Disconnection Command	"leave.” (Valid)	User removed from ConcurrentHashMap, socket closed, thread freed to pool.	Program exited, but the thread remained	**Redesigned** ClientHandle to use a finally

			permanently stuck in the server pool.	block for socket.close()
Command Input	/Kick testUser" (Illegal)	Sent by a standard client. System should deny the request.	"Permission denied. You do not have admin rights."	n/a
Command Input	"/kick testUser" (Valid)	Sent by the 'admin' account. Target user is forcefully disconnected.	"Successfully kicked testUser". Target user thread ended	n/a
Private Message Input	"/msg test2 Hello" (Valid)	Message routed exclusively to 'test2' and not broadcasted to others.	Message received only by 'test2' as "(Private) admin whispers: Hello".	n/a
Cryptographic Salt	16-byte array (Valid)	Salt is stored in the database alongside the hashed password.	Traceback error: Writing raw bytes to Users.csv caused file corruption.	**Redesigned** Code to now use Base64.getEncoder().encodeToString()
User Password Storage	"password 123" *(Valid data)*	password is not stored in plaintext. It is hashed using SHA-256 before being saved to the database.	Password successfully hashed. Database shows Base64 ciphertext instead of plain text	n/a
Message payload encryption	"Hello World" *(Valid data)*	The message is converted to AES Base64 ciphertext before leaving the client socket. Intercepted data is unreadable.	**Untested.** The encryption logic compiles successfully, but network-level packet interception couldn't be done	n/a

4.2 Evidence of Testing

The visualisation of the tests outlines in the test data table above has been compiled in Appendix A: Testing Evidence to maintain document readability. These screenshots provide definite proof of the systems operational security and concurrency:

Appendix A, Figure 1: Cryptographic Storage (Data Integrity) demonstrating SHA-256 hashing in Users.csv.

Appendix A, Figure 2: Defensive Account Lockout demonstrating the boundary trigger for failed authentication.

Appendix A, Figure 3: CSV Data Sanitization demonstrating the interception of illegal characters in ChatLogs.csv.

Appendix A, Figure 4: Concurrent Private Routing demonstrating parallel messaging isolation across multiple client threads.

Appendix A, Figure 5: Administrative Privilege Enforcement demonstrating identity-based access control for the kick command.

4.3 Meeting initial Requirements

To ensure that the system met the core objectives and requirements outlined in section 1.3 the test plan shown above in section 4.1 was solely based of the initial requirements and features of the system. Ensuring that no core requirement was left untested. Some of those being:

- Multi-Client Architecture: validated by the test data 'Active User Check' and visual evidence of simultaneous connections on the server terminal and client terminal.
- Confidentiality and data integrity: validated by the test data 'Cryptographic salt byte testing and chat message csv writing.'
- Security: Validated by test data 'Failed Login Counter' and 'Admin Kick command'

4.4 Evaluation + Reflection

4.4.1 System Evaluation

The final implementation successfully solves the problem of network communication concurrently. The integration of the ExecutorService and the ConcurrentHashMap proved to be a good design choice and highly resilient , during testing the system managed multiple simultaneous connections and private messages without thread locking, resource exhaustion and system crashing. Furthermore, the implementation of the double security layer , using SHA-256 with salt for user authentication and AES-128 for message payload encryption , successfully met the system requirements of confidentiality and data integrity.

4.4.2 User Experience

Throughout the development of the system, one major focus along with the security of the system was the user experience. Implementing the background listener 'Thread listener = new Thread(new Runnable()' in Client1.java avoids the freezing problem between users typing and listening for new messages. Using this method this allowed for users to type and receive messages concurrently. In addition, defence logic was used to provide the users with clear feedback from the system such as 'ACCOUNT LOCKED' or 'Invalid Username' or 'Commas not allowed

' to ensure a smooth user experience, rather than the system crashing silently and leaving the users confused.

4.5 System Limitations

1. **Inability to perform live packet interception**: While the AES encryption logic was successfully implemented, a limitation of the testing phase was the inability to perform live packet interception (sniffing) to definitively prove the ciphertext's unreadability in transit. In a future iteration, utilizing tools like Wireshark would formally verify this constraint.

2. **Reliance on Hardcoded Key** : Currently the system relies on a hard coded encryption key: which is shared between the client and the server (CourseworkKey123) as a future improvement I would consider implementing an asymmetric encryption (RSA) rather than a symmetric (AES) where the client and the server could exchange when the server first connects as asymmetric schemes can be more secure for key distribution (Kapoor, Thakur).
3. **Managing Threadpool saturation:** currently the server has a maximum Threadpool of 10 clients, which prevents resource exhaustion, currently if an additional client tries to join, they are just waiting in the terminal for a thread to be freed by a client leaving. For a future improvement alternatively the client waiting would be put onto an additional socket but forced to wait until a thread frees up to have access to the server and be presented with a graceful error message such as “Client pool full. You are in the waiting list; there are x number of users ahead of you.” Which improves the user experience at peak server times.
4. **Vulnerability to chat flooding**: Currently the system processes incoming messages using a while loop with no restriction in how fast a user can send messages, if a malicious actor utilised an automated script to send hundreds of messages per second, it would forces the server to lag by constantly triggering route group message method. As a future improvement of the system rate limiting could be implemented by recording the users timestamp on their most recent message and dropping any new messages within 500milli seconds of the previous one to avoid this vulnerability.

5.0 References

Abbas, W., Joshua, S., Abbas, A., and Lee, J. (2025). An End-to-End GSM/SMS Encrypted Approach for Smartphone Employing Advanced Encryption Standard(AES). *2026 20th International Conference on Ubiquitous Information Management and Communication (IMCOM)*, pp. 1-8. Doi: 10.1109/imcom69009.2026.11360886.

Olakanmi, O. and Odeyemi, K. (2021). Throttle: An efficient approach to mitigate distributed denial of service attacks on software-defined networks. *Security and Privacy*, 4. Doi: 10.1002/spy2.158.

Patra, R., and Patra, S. (2021). Cryptography: A Quantitative Analysis of the Effectiveness of Various Password Storage Techniques. *Journal of Student Research*. Doi: 10.47611/jsrhs.v10i3.1764.

Karacali, H., Donum, N. and Cebel, E. (2024). Cryptographic Enhancement of Named Pipes for Secure Process Communication. *The European Journal of Research and Development*. Doi: 10.56038/ejrnd.v4i2.428.

Yinbo, Z. (2012). Thread Pool in the Concurrent Server. *Computer and Digital Engineering*.

Ali, A., Ahmed, M., and Khan, A. (2021). Audit Logs Management and Security - A Survey. *Kuwait Journal of Science*. Doi: 10.48129/kjs.v48i3.10624.

Saakashvili, E. (2025). Validation and Sanitization of User-Entered Data for Security Purposes. *Proceedings of "Computer Science and Information Technologies" International Conference*. Doi: 10.51408/csit2025_31.

https://www.w3schools.com/java/java_ref_hashmap.asp

https://happycoding.io/tutorials/java-server/sanitizing-user-input#sanitizing-user-input

Kapoor, J., and Thakur, D. (2022). Analysis of Symmetric and Asymmetric Key Algorithms. *ICT Analysis and Applications*. Doi: 10.1007/978-981-16-5655-2_13.

Oracle (2026) "Java Cryptography Architecture (JCA) Reference Guide - javax.crypto

6.0 Appendix

Appendix A

Evidence 1: Cryptographic Storage (Data Integrity)

Users.csv > data

```
1  toby,gEKd+Sqjw9uwvwVtZxzmc2zB7/ekDCHilwfPdiEpPmk=,hULxpmoiZbkrBGCSegATnA==
2  Test1,qluTUp7E0HQKbb8dkhq7m3bGwVoIfWce9rp1zJ5NreM=,MSU/5dVKzEsTdKNDpy5wpw==
3  admin,OB9ExHmLODNIybnUWkicJ5JOI0k0kZx15JJ/UTOf3+E=,O1toIxiu7GcenZKXE0QgHg==
4  Test2,pK1bU8tAnkbEAOInGbjP8ODnYvD5ZrghgXa3CoGuhjw=,au67+oFjtBD7WYdgFxtNEw==
5  Test3,Tiypn+YMRDF7kiFsQKqYKJHQthyNUPqZ7DgrdvDzXSU=,3u39jZNMm7I6p/vfjRacVg==
6  Test4,0JT4T3W0LhU8hnYDwMl2TV0vA18yTy+7oKxadnLLVcA=,CGOayw+UqyHSSZLesrUFlw==
7  Test6,D/PWWbvfCMjz8J9uw9dmtzNmthuec0cNBIXbJBlIYO0=,1HPlurmekP+T/n+SnAjolQ==
8  Test7,vnj6pKuSGTdcY9qEXNZO2W6wOvVAlms6ngI2uWDJilE=,mXBvX8iTbmHJseDujU6d+w==
9
```

Figure 1: The secure user database. As demonstrated, the plaintext passwords are not stored. The system successfully utilizes SHA-256 hashing and Base64 encoding to store secure ciphertext and unique cryptographic salts, meeting the confidentiality requirement.

Evidence 2: Defensive Account Lockout

```
Welcome! Type 1 to login or 2 to Register:
1
Enter your Username:
Test1
Enter your password:
Test2
Authentication failed:ACCOUNT_LOCKED
```

Figure 2: Boundary testing of the authentication gateway. After three failed attempts, the system successfully triggers the defensive counter logic, denying further access and protecting the network from brute-force attacks.

Evidence 3: CSV Data Sanitization

ChatLogs.csv > data

```
1  Sender,Reciever ,Content,TimeStamp
2  System,All,Test1 has joined the server.,2026-04-30 15:21:34
3  System,All,Test2 has joined the server.,2026-04-30 15:21:53
4  System,All,Test3 has joined the server.,2026-04-30 15:22:09
5  Test1,All,Hello Everyone,2026-04-30 15:22:43
6  Test1,Test3,Hello Test3,2026-04-30 15:23:34
7  System,All,admin has joined the server.,2026-04-30 15:24:09
8  admin,Test3,ADMIN KICK EXECUTED,2026-04-30 15:24:42
9  System,All,Test3 has disconnected.,2026-04-30 15:24:42
```

Figure 3: The system audit trail. This confirms that the LogManager successfully intercepts and sanitizes illegal comma characters, replacing them with spaces to prevent CSV corruption.

Evidence 4: Concurrent Private Routing

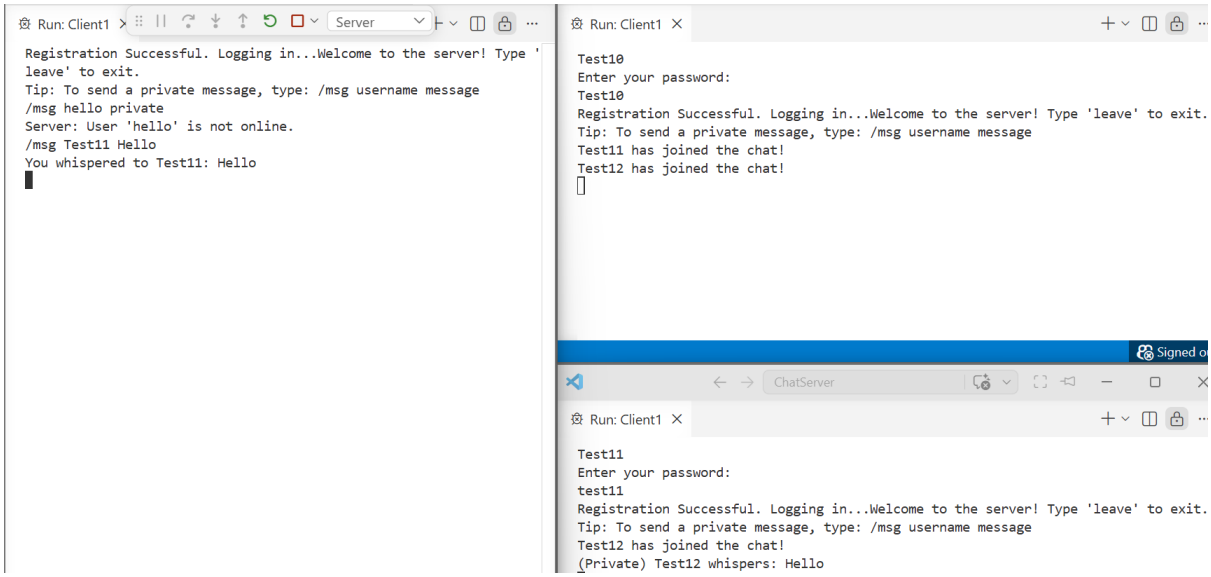

Figure 4: Parallel messaging logic in action. The server's targeted routing successfully isolates the private whisper, ensuring that the payload is only delivered to the specific thread associated with the target user.

Evidence 5: Administrative Privilege Enforcement

```
Test11
Enter your password:
test11
Registration Successful. Logging in...Welcome to the server! Type 'leave' to exit.
Tip: To send a private message, type: /msg username message
Test12 has joined the chat!
/kick Test12
Server: Permission denied. You do not have admin rights.
```

Figure 5: Identity-based access control. The system successfully intercepts the restricted command and evaluates the thread's username, denying the action to non-administrators to maintain server stability.